

Proving the hard-to-round search

Code: `code/APaul/htr.c` and `code/APaul/htrplan/`. Numbers are marked **read** (off the code), **measured** (a run) or **computed**.

1 Definitions

- e is the exponent of $\exp x$: $2^{e-1} \leq \exp x < 2^e$.
- $Y(x) = \exp(x) \cdot 2^{54-e}$, in $[2^{53}, 2^{54})$. The integers of this scale are the doubles and the midpoints between them: the **grid**.
- x is **hard to round at level m** when $\text{dist}(Y(x), \mathbb{Z}) < 2^{-m}$: the m bits after the round bit of $\exp x$ are all equal.
- u is the ulp of x : the distance from x to the next double.

2 The theorem

For an interval $[x_0, x_1)$ and a level m :

$$(T) \quad \forall x \in [x_0, x_1), x \text{ hard to round at level } m \Rightarrow x \text{ is printed.}$$

The program may print false cases, never miss a true one.

3 How the search works

The search cuts $[x_0, x_1)$ into **chunks** of $n = 2^{20}$ consecutive doubles $x + iu$, $0 \leq i < n$. On a chunk, Y is close to a **line**, since the derivative of $\exp$ is $\exp$:

$$Y(x + iu) \approx Y(x) + i \cdot Y(x)u.$$

Only the fractional parts matter. They are kept as 64-bit integers (the unit is 2^{-64}):

- $A \approx$ the fractional part of $Y(x)$, $B \approx$ that of the slope $Y(x)u$;
- the **drift** is how far Y moves away from the line over the chunk, at most $Y(x)u^2n^2/2$;
- the **window** is $E = 2^{64-m} + \text{drift}$.

The inner loop adds B to A n times. Each i where A is within E of an integer is a **candidate**: it is passed to **report**, which checks it with a precise evaluation and prints it if it is hard.

Why it works. If $x + iu$ is hard, Y is within 2^{-m} of an integer, so the line is within $2^{-m} + \text{drift} + \text{errors of it}$. If the window covers that, i is a candidate.

4 The three programs

- **Program 1**, `htr.c`: evaluates $\exp$ with MPFR in three functions. Not proved; it is the reference (Section A).
- **Program 2**, the generic search: `htr.c` with the three functions as parameters `eval`, `expo`, `report`, which must meet a contract E1–E4. Proved once, for all parameters that meet it. With MPFR's functions as parameters, it is Program 1 again.
- **Program 3**: parameters for Program 2 in integer arithmetic, no MPFR, proved to meet E1–E4.

5 The contract and the dependencies

The contract. `eval(x)` returns $h + l + s \approx \exp x$ (three doubles); `expo(x)` returns an integer; `report` calls `check(x, m)`, which returns 0 to reject a candidate.

	requirement	used by
E1	$ h + l + s - \exp x \leq 2^{-\kappa} \exp x$	2b

E2	$ l < 2^{e-53}$ and $ s < 2^{e-54}$	2a
E3	$2^{\text{expo}(x)-1} \leq \exp x < 2^{\text{expo}(x)}$	2a
E4	$\text{dist}(Y(x), \mathbb{Z}) < 2^{-m} \Rightarrow \text{check}(x, m) \neq 0$	(T)

κ is the number of correct bits of **eval**. The code of Program 2 does not fix it; the proof of 2b does, since **eval**'s error is one of the errors the window must cover. About 118 is enough (**computed**): **eval**'s error is then below the last bit of **A**. Program 3 gives 158.

The dependencies. Each line needs the lines below it.

```
(T) every hard x in [x0, x1) is printed
|-- (S) the search passes every hard x to report          needs E1 E2 E3
|   |-- 2a the conversions to 64 bits          needs E2 E3    to do
|   |-- 2b the window covers the errors        needs E1       to do, the hard point
|   |-- 2c the inner loop                      done, Scan.v
|   |-- 2d the chunks cover [x0, x1)          done, ScanAll.v
|-- report prints every hard x it is given          needs E4

E1..E4 for Program 3
|-- (E) the integer y is within 2^-160 of exp x          to do
|-- 3a (E) + the split of y                            => E1 with kappa = 158, E2
|-- 3b (E) + an assert on y                            => E3
|-- 3c (E) + the distance test                        => E4
```

6 Program 2: the generic search

Interface, search.h:

```
// Program 2, the generic search: htr.c with the evaluator as a parameter.
//
// x is hard to round at level m when  $\text{dist}(\exp(x) 2^{(54-e)}, \mathbb{Z}) < 2^{-m}$ ,
// where e is the exponent of exp(x) (as frexp gives it). The theorem:
// (S) for every double x in [x0, x1) hard to round at level m,
// search calls report (x, m),
// provided eval and expo meet E1 to E3 below.

#include <stdint.h>

// eval (&h, &l, &s, x) approximates exp(x) by h + l + s:
// (E1)  $|h + l + s - \exp(x)| \leq 2^{-\text{kappa}} * \exp(x)$ 
// (E2)  $|l| < 2^{(e-53)}$  and  $|s| < 2^{(e-54)}$ 
// kappa is the accuracy the evaluator must reach. Its value is fixed by
// the proof of the window: about 118 is enough (computed, not proved).
// E2 lets the search read the bits after the round bit from l and s alone.
typedef void (*eval_t) (double *h, double *l, double *s, double x);

// expo (x) returns e, the exponent of exp(x):
// (E3)  $2^{(\text{expo}(x) - 1)} \leq \exp(x) < 2^{\text{expo}(x)}$ 
// The search scales h, l, s by  $2^{(54-e)}$ , and checks that e is the same
// at both ends of [x0, x1).
typedef int (*expo_t) (double x);

// report (x, m) is called on every candidate x. S is about these calls.
// A report that filters the candidates with a check (x, m), returning 0
// to reject, must keep every hard x:
```

```
// (E4)  $\text{dist}(\exp(x) 2^{(54-e)}, Z) < 2^{-m} \Rightarrow \text{check}(x, m) \neq 0$ 
typedef void (*report_t) (double x, int m);

void search (eval_t eval, expo_t expo, report_t report,
             double x0, double x1, int m);
```

Code: htr.c's search with four changes (make search.diff):

```
--- htr.c
+++ search.c
@@ -10,9 +10,10 @@
     return ~ret + (uint64_t) 1;
 }
// search hard-to-round cases of exp in [x0,x1)
-// with at least m0 identical bits after round bit
-static void
-search (double x0, double x1, int m)
+// with at least m identical bits after round bit
+void
+search (eval_t eval, expo_t expo, report_t report,
+       double x0, double x1, int m)
{
    uint64_t n = 1ul << 20;
    int e, e0, e1;
@@ -25,17 +26,14 @@
    double ux = ldexp (1.0, e0 - 53); // ux = ulp(x)

    // check exp(x) lies in the same binade in [x0, x1)
-   double y0 = ref_exp (x0);
-   double y1 = ref_exp (nextafter (x1, x0));
-   frexp (y0, &e);
-   frexp (y1, &e1);
+   e = expo (x0);
+   e1 = expo (nextafter (x1, x0));
    assert (e == e1);
-   double uh = ldexp (1.0, e - 53); // uh = ulp(exp(x))

    double h, l, s;
    double x = x0;
    while (x < x1) {
-       dd_exp (&h, &l, &s, x);
+       eval (&h, &l, &s, x);
        double dh = h * ux, dl = l * ux; // 1st derivative, multiplied by ux
        double dd = h * ux * ux; // 2nd derivative, multiplied by ux^2

@@ -65,7 +63,7 @@
    for (uint64_t i = 0; i < n; i++) {
        if (__builtin_expect (A < 2*E, 0)) { // found potential hard-to-round case
            double xi = x + i * ux;
-           check (xi, m);
+           report (xi, m);
        }
        A += B;
    }
```

Obligations.

- **2a.** h, l, s are scaled by 2^{54-e} and cut into 64-bit integers (`get_uint64`), with sums modulo 2^{64} . This reads the right bits only if e is exact (E3) and l, s are small (E2).
- **2b.** The code's window is $2^{64-m} + \text{drift}$ (`read`). Four errors are not in it: `eval`'s (κ); the drift uses the curvature at the start of the chunk, not its maximum; the truncations of **B** add up over n steps; **A** is truncated. The proof must show they fit in the margin, or add a term to E.
- **2c, 2d.** Done: `Scan.v`, `ScanAll.v`.

7 Program 3: the proved evaluator

`eval_fix.c` provides `eval_fix`, `expo_fix` and `check_fix` (the check in `report`). All three start from `fix_exp`, which computes an integer y and an integer k with $\exp x \approx y \cdot 2^{k-200}$, by Tang's method:

1. write $x = N \ln 2/256 + r$, with N an integer and $|r| \leq 2^{-9.5}$; then $N = 256k + j$ with $0 \leq j < 256$;
2. compute $\exp r$ with its Taylor polynomial of degree 13;
3. multiply by $2^{j/256}$, taken from a table of 256 constants; then $\exp x = 2^k \cdot 2^{j/256} \cdot \exp r$.

Every quantity is an integer Z standing for $Z \cdot 2^{-200}$. `check_fix` answers hard, not hard, or **undecided** when the error of y does not allow to decide; undecided cases are printed. Code: Section B.

The one bound:

$$(E) \quad |y \cdot 2^{k-200} - \exp x| \leq 2^{-160} \cdot 2^k.$$

From (E) to E1–E4.

- **3a.** h, l, s are the first three blocks of 53 bits of y : they lose less than 2^{-158} relative (E1, $\kappa = 158$), and each is below the last bit of the one before (E2).
- **3b.** The code asserts that y is not within its error of a power of 2, so y and $\exp x$ have the same exponent (E3).
- **3c.** `check_fix` answers “not hard” only if the distance of y to the grid, minus 2^{-160} , is still at least 2^{-m} (E4).

Proof of (E). One lemma per step of `fix_exp`. Errors in units of 2^{-200} , **computed**:

step	error	proof
r	2 units	$\ln 2/256$ certified by <code>interval</code>
$ r \leq 2^{-9.5}$	—	an <code>assert</code> in the code
Horner	about 4 units	one floor per step
Taylor remainder	2^{-169}	one <code>interval</code> call
$\times 2^{j/256}$	1.5 units	256 constants, each certified by <code>interval</code>
total	$\leq 2^{-160}$	8 bits to spare

8 What to do

1. 2a and 2b: proves (S) for any parameters, and fixes κ .
2. (E) and 3a–3c: Program 3 meets the contract.
3. The whole range: Program 2 only works for $|x| < 1/2$, one binade at a time, and above $x = -634$ (Section D).
4. Tie the Rocq models to the C (or its Capla port): not tried yet.

A Annex: Program 1, `htr.c`

MPFR is used in `dd_exp` (`eval`), `ref_exp` (`expo`) and `check` (in `report`).

```

// search hard-to-round cases of exp in [0.25,0.5)
// with at least 43 identical bits after round bit

#include <stdio.h>
#include <stdint.h>
#include <mpfr.h>
#include <assert.h>
#include <math.h>

// reference code using MPFR, with RNDZ
double
ref_exp (double x)
{
    mpfr_t y;
    mpfr_exp_t emin = mpfr_get_emin ();
    mpfr_set_emin (-1073);
    mpfr_init2 (y, 53);
    mpfr_set_d (y, x, MPFR_RNDN);
    int inex = mpfr_exp (y, y, MPFR_RNDZ);
    mpfr_subnormalize (y, inex, MPFR_RNDZ);
    double ret = mpfr_get_d (y, MPFR_RNDN);
    mpfr_clear (y);
    mpfr_set_emin (emin);
    return ret;
}

// put in h+l+s a triple-double approximation of exp(x)
static void
dd_exp (double *h, double *l, double *s, double x)
{
    mpfr_t t;
    mpfr_init2 (t, 3*53+2);
    mpfr_set_d (t, x, MPFR_RNDN);
    mpfr_exp (t, t, MPFR_RNDN);
    *h = mpfr_get_d (t, MPFR_RNDN);
    mpfr_sub_d (t, t, *h, MPFR_RNDN);
    *l = mpfr_get_d (t, MPFR_RNDN);
    mpfr_sub_d (t, t, *l, MPFR_RNDN);
    *s = mpfr_get_d (t, MPFR_RNDN);
    mpfr_clear (t);
}

unsigned long checks = 0, found = 0;

static void
check (double x, int m)
{
    // printf ("check x=%la\n", x);
    checks ++;
    mpfr_t y, z;
    mpfr_init2 (y, 54);
    mpfr_init2 (z, 53 + m);
    mpfr_set_d (y, x, MPFR_RNDN);
    mpfr_exp (z, y, MPFR_RNDN);
    mpfr_set (y, z, MPFR_RNDN);
    if (mpfr_cmp (y, z) == 0) {

```

```

    printf ("%la\n", x);
    found ++;
}
mpfr_clear (y);
mpfr_clear (z);
}

// given x, -1 < x < 1, return the 64-bit value corresponding
// to the fractional part of x
static uint64_t
get_uint64 (double x)
{
    assert (-1 < x && x < 1);
    x = ldexp (x, 64);
    if (x >= 0) return x;
    uint64_t ret = (uint64_t) (-x);
    return ~ret + (uint64_t) 1;
}

// search hard-to-round cases of exp in [x0,x1)
// with at least m0 identical bits after round bit
static void
search (double x0, double x1, int m)
{
    uint64_t n = 1ul << 20;
    int e, e0, e1;

    // check ulp(x0) = ulp(nextbelow(x1))
    frexp (x0, &e0);
    frexp (nextafter (x1, x0), &e1);
    assert (e0 == e1);

    double ux = ldexp (1.0, e0 - 53); // ux = ulp(x)

    // check exp(x) lies in the same binade in [x0, x1)
    double y0 = ref_exp (x0);
    double y1 = ref_exp (nextafter (x1, x0));
    frexp (y0, &e);
    frexp (y1, &e1);
    assert (e == e1);
    double uh = ldexp (1.0, e - 53); // uh = ulp(exp(x))

    double h, l, s;
    double x = x0;
    while (x < x1) {
        dd_exp (&h, &l, &s, x);
        double dh = h * ux, dl = l * ux; // 1st derivative, multiplied by ux
        double dd = h * ux * ux; // 2nd derivative, multiplied by ux^2

        // prepare loop
        // A is the bits of h+l after the round bit
        h = ldexp (h, 54 - e); // now ulp(h) = 2
        l = ldexp (l, 54 - e);
        // we now have -2 < l < 2
        if (l >= 1.0) l -= 1.0;
        while (l < 0) l += 1.0;
    }
}

```

```

assert (0 <= l && l < 1);
s = ldexp (s, 54 - e);
uint64_t lu = get_uint64 (l), su = get_uint64 (s);
uint64_t A = lu + su;
// scale the derivatives too
dh = ldexp (dh, 54 - e);
dl = ldexp (dl, 54 - e);
dd = ldexp (dd, 54 - e);
uint64_t B = get_uint64 (dh) + get_uint64 (dl);
// the maximal error is bounded by dd/2*n^2
dd = dd / 2.0 * n * n;
// hard-to-round cases are at distance < 2^-(m+1) ulp(h)
uint64_t E = get_uint64 (dd) + (1ul << (64 - m));
// we want to find values of A in [-E, E]
A += E; // shift A
// now we want to find values of A in [0,2*E]
for (uint64_t i = 0; i < n; i++) {
    if (__builtin_expect (A < 2*E, 0)) { // found potential hard-to-round case
        double xi = x + i * ux;
        check (xi, m);
    }
    A += B;
}
x += n * ux;
if (x > x1)
    n = (x1 - x) / ux;
}
}

/* With x0 = 0.25, x1 = 0.25001, m = 35
on thym.loria.fr (Intel(R) Core(TM) Ultra 7 265) with gcc 15.3.0
(-O3 -march=native):

0x1.00002385331bep-2
0x1.0000561fc7e06p-2
0x1.0001cd08ef3d4p-2
0x1.00026c769e211p-2
0x1.00028a78a7303p-2
7056503 checks, found 5 hard-to-round

real 0m39.080s
user 0m39.032s
sys 0m0.008s

BaCSeL:
make GMP=$GMP MPFR=$MPFR FPLLL=$FPLLL DEFSAL="-DEXP -DBASIS=2 -DAUTOMATIC"
zimmerma@thym:~/svn/bacsel$ ./bacsel -rnd_mode all -prec 128 -n 53 -nn 53 -m 36 -t
18 -t0 4503599627370496 -t1 4503779771355591 -d 2 -alpha 2 -e_in -2 -nthreads 4
zimmerma@thym:~/svn/bacsel$ ./bacsel -rnd_mode all -prec 128 -n 53 -nn 53 -m 36 -t
18 -t0 4503599627370496 -t1 4503779771355591 -d 2 -alpha 2 -e_in -1 -nthreads 4
./bacsel -rnd_mode all -prec 128 -n 53 -nn 53 -m 36 -t 18 -t0 4503599627370496 -t1
4503779771355591 -d 2 -alpha 2 -e_in -1 -nthreads 4
Welcome to BaCSeL version 4.0, using 4 thread(s)
Compilation flags: -DBASIS=2 -DEXP -DAUTOMATIC
LLL parameters: delta=0.999 eta=0.501
Parameters for AUTOMATIC: FAILURE_LOWERBOUND=2.0 FAILURE_UPPERBOUND=4.0

```

```

*** x=0x4.0001587f1f818p-4, distance is: 1.059463030e-11 0x1.0000561fc7e06p-2
*** x=0x4.00008e14cc6f8p-4, distance is: 1.055287224e-11 0x1.00002385331bep-2
*** x=0x4.00073423bcf5p-4, distance is: 2.577853443e-12 0x1.0001cd08ef3d4p-2
*** x=0x4.000a29e29cc0cp-4, distance is: 7.860279955e-12 0x1.00028a78a7303p-2
*** x=0x4.0009b1da78844p-4, distance is: 7.913797859e-12 0x1.00026c769e211p-2
*/
int main()
{
    double x0 = 0.25, x1 = 0.25001;
    int m = 35;
    search (x0, x1, m);
    printf ("%lu checks, found %lu hard-to-round\n", checks, found);
}

```

B Annex: Program 3

```

// The parameters of eval_fix.c.

#include <gmp.h>

#define FIX_P 200 // bits after the point of every fixed-point number
#define FIX_TAB 256 // table size: x = (256 k + j) ln2/256 + r
#define FIX_LG 32 // extra bits of ln2/256, so that N * ln2/256 is exact
// to 2^-FIX_P for |N| < 2^32
#define FIX_DEG 13 // degree of the Taylor polynomial of exp(r)

// Program 3, the proved evaluator: exp with integers only (GMP's mpz).
// Tang's scheme:
// x = N ln2/256 + r, N = 256 k + j, 0 <= j < 256, |r| <= 2^-9.5
// exp(x) = 2^k * 2^(j/256) * exp(r)
// A fixed-point number is an integer X standing for X / 2^FIX_P.
// Designed error (not yet proved): 2^-169 on y, 2^-158 after the split.

#include <assert.h>
#include "fix.h"
#include "eval.h"
#include "fix_table.h"

static mpz_t T[FIX_TAB]; // 2^(j/256), rounded to nearest
static mpz_t ln2; // ln2/256, with FIX_LG extra bits
static mpz_t C[FIX_DEG + 1]; // 1/i!, rounded down
static mpz_t rmax; // bound on |r|: 181/256 2^-9 < 2^-9.5
static mpz_t err; // bound on the error of y: 2^-160

void
fix_init (void)
{
    for (int j = 0; j < FIX_TAB; j++)
        mpz_init_set_str (T[j], fix_T[j], 16);
    mpz_init_set_str (ln2, fix_ln2, 16);
    mpz_t f;
    mpz_init_set_ui (f, 1);
    for (int i = 0; i <= FIX_DEG; i++) {
        if (i > 0) mpz_mul_ui (f, f, i);
        mpz_init (C[i]);
    }
}

```

```

    mpz_setbit (C[i], FIX_P);
    mpz_fdiv_q (C[i], C[i], f);
}
mpz_clear (f);
mpz_init_set_ui (rmax, 181);
mpz_mul_2exp (rmax, rmax, FIX_P - 17);
mpz_init (err);
mpz_setbit (err, FIX_P - 160);
}

// exp(x) = y / 2^FIX_P * 2^k; returns k
static long
fix_exp (mpz_t y, double x)
{
    mpz_t X, r, t;
    mpz_inits (X, r, t, NULL);
    // X = x 2^FIX_P, exactly
    int ex;
    double mx = frexp (x, &ex);
    long sh = ex - 53 + FIX_P;
    assert (sh >= 0);
    mpz_set_si (X, (long) ldexp (mx, 53));
    mpz_mul_2exp (X, X, sh);
    // r = x - N ln2/256, checked small
    long N = (long) nearbyint (x * (FIX_TAB / M_LN2));
    mpz_mul_si (t, ln2, N);
    mpz_fdiv_q_2exp (t, t, FIX_LG);
    mpz_sub (r, X, t);
    mpz_abs (t, r);
    assert (mpz_cmp (t, rmax) < 0);
    // exp(r) by Horner's rule, one truncation per product
    mpz_set (y, C[FIX_DEG]);
    for (int i = FIX_DEG - 1; i >= 0; i--) {
        mpz_mul (y, y, r);
        mpz_fdiv_q_2exp (y, y, FIX_P);
        mpz_add (y, y, C[i]);
    }
    // times 2^(j/256)
    long j = ((N % FIX_TAB) + FIX_TAB) % FIX_TAB;
    mpz_mul (y, y, T[j]);
    mpz_fdiv_q_2exp (y, y, FIX_P);
    // y is not within err of a power of 2, so its exponent is that of exp(x)
    mpz_sub (t, y, err);
    size_t nb = mpz_sizeinbase (t, 2);
    mpz_add (t, y, err);
    assert (mpz_sizeinbase (t, 2) == nb);
    mpz_clears (X, r, t, NULL);
    return (N - j) / FIX_TAB;
}

int
expo_fix (double x)
{
    mpz_t y;
    mpz_init (y);
    long k = fix_exp (y, x);

```

```

    int e = (int) mpz_sizeinbase (y, 2) - FIX_P + k;
    mpz_clear (y);
    return e;
}

// the top 53 bits of y, as a double scaled by 2^sc; y keeps the rest
static double
top53 (mpz_t y, long sc)
{
    if (mpz_sgn (y) == 0) return 0.0;
    long sh = (long) mpz_sizeinbase (y, 2) - 53;
    if (sh < 0) sh = 0;
    mpz_t hi;
    mpz_init (hi);
    mpz_fdiv_q_2exp (hi, y, sh);
    double d = ldexp (mpz_get_d (hi), sh + sc);
    mpz_mul_2exp (hi, hi, sh);
    mpz_sub (y, y, hi);
    mpz_clear (hi);
    return d;
}

void
eval_fix (double *h, double *l, double *s, double x)
{
    mpz_t y;
    mpz_init (y);
    long k = fix_exp (y, x);
    *h = top53 (y, k - FIX_P);
    *l = top53 (y, k - FIX_P);
    *s = top53 (y, k - FIX_P);
    mpz_clear (y);
}

// Is x hard to round at level m: is exp(x) 2^(54-e) within 2^-m of an
// integer? y has nb bits; its top 54 are the integer part, the other
// nb - 54 bits the fraction. Returns 1 (hard), 0 (not hard), or 2 when
// the error of y does not allow to decide; a caller that builds a
// superset keeps the undecided ones.
int
check_fix (double x, int m)
{
    mpz_t y, low, d, t;
    mpz_inits (y, low, d, t, NULL);
    fix_exp (y, x);
    long f = (long) mpz_sizeinbase (y, 2) - 54; // bits of the fraction
    mpz_fdiv_r_2exp (low, y, f); // the fraction, times 2^f
    mpz_set_ui (d, 0);
    mpz_setbit (d, f);
    mpz_sub (d, d, low); // distance to the next integer
    if (mpz_cmp (low, d) < 0) mpz_set (d, low); // d = the distance, times 2^f
    mpz_set_ui (t, 0);
    mpz_setbit (t, f - m); // the threshold 2^-m, times 2^f
    int r;
    mpz_add (low, d, err);
    if (mpz_cmp (low, t) < 0) r = 1;
}

```

```

else {
    mpz_sub (low, d, err);
    r = mpz_cmp (low, t) >= 0 ? 0 : 2;
}
mpz_clears (y, low, d, t, NULL);
return r;
}

```

The constants come from `gen_fix.c` (with MPFR); they are not trusted, since each is certified.

C Annex: the test

`make test` compares, line by line, the candidates and the hard cases of `htr.c`, Program 2 with MPFR, and Program 2 with Program 3 (`htr3`, linked without MPFR). **Measured**, 2026-09-28:

slice, 10 000 chunks	candidates	result
from 0.25 up	410 325	identical; one hard case; none undecided
from -0.25 down	498 078	identical; one hard case; none undecided

There are many candidates because the drift, about 2^{48} units, is much wider than the target 2^{29} (**computed**): about 41 per chunk, as in `htr.c`'s comment (**read**).

D Annex: limits

- `get_uint64` needs the slope below 1: $|x| < 1/2$ (**computed**). Fix: keep only its fractional part.
- $[x_0, x_1)$ must sit in one binade of x and of $\exp x$.
- Below $x \approx -708$, $\exp x$ is subnormal; above 709.78, it overflows.
- Three doubles hold 158 bits only for $x \geq -634$ (**computed**); at $x = -700.3$ they keep 66 (**measured**). Below, Program 2 must take y itself.
- `eval` is called once per chunk, 0.7 ms of inner loop (**computed**), so its speed does not matter; the whole range is about 12 core-years (**computed**).